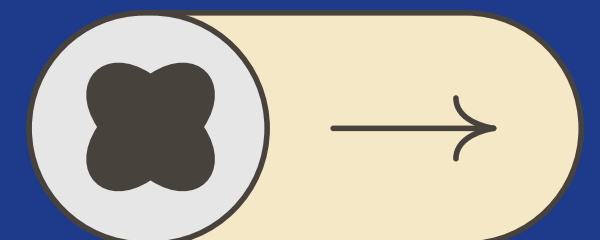


FPGA-ACCELERATED AUDIO FINGERPRINTING USING HLS ON THE KRIA KV260

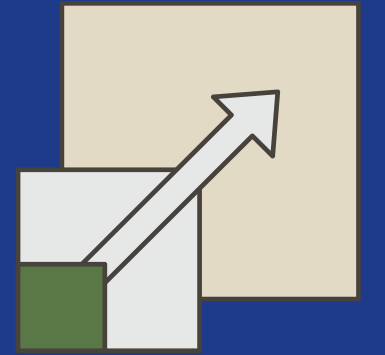
Team Members:

- Venus Baranwal:
z5483461
- Rukhsaar Aujla:
z5619382
- Esha Tripathi:
z5391046
- Netik Kumar Maheshwari:
z5636903

FPGA-Accelerated Audio Fingerprinting using HLS on the Kria KV260

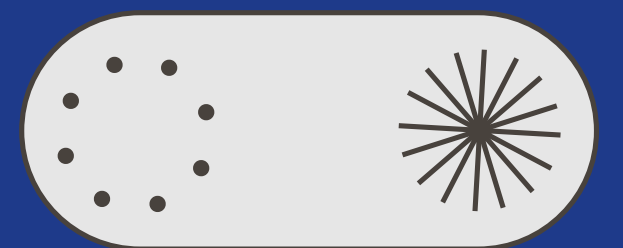


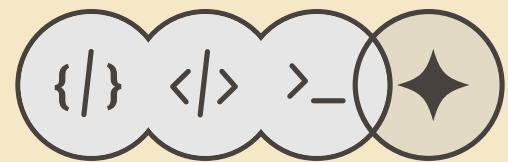
PROBLEM OUTLINE



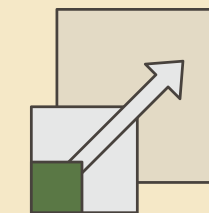
Can audio fingerprint generation be accelerated using an FPGA?

- Audio fingerprinting processes thousands of audio samples to generate a compact signature.
- The algorithm we're investigating is a c++ port of dejavu (a popular audio fingerprinting algorithm, originally in python)
- It repeatedly performs FFTs and local peak detection.
- These stages exhibit parallelism, making them promising candidates for hardware acceleration.
- We aim to accelerate these hotspots using HLS pragmas and evaluate the performance gains achievable on the Kria KV260.



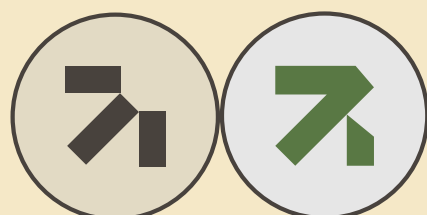


HOW IT WORKS



1. **Input Audio:** Receive raw audio samples
2. **Windowing:** Split into overlapping frames
3. **FFT:** Convert to frequency domain
4. **Spectrogram:** Time-frequency representation
5. **Peak Detection:** Identify prominent frequency peaks
6. **Hash Generation:** Encode peaks into hashes
7. **Audio Fingerprint:** Compact audio signature

FPGAs are notoriously bad at handling strings!
So, we won't bother with optimising the SHA
generation stuff. :)



PROFILING STATS

Kernel	Timing
Hashing_and_JSON:	4.80104 ms
Spectrogram_Math:	6.89483 ms
DFT	1.54862 ms
Peak_Detection	18.8248 ms
Mat Conversion	3.81288 ms
Windowing	11.192 ms

We can see from the prelim profiling timings on CPU that 40% of runtime is spent in peak detection.

- Strategy:
- Accelerate Peak Detection & Streaming: Move windowing, DFT, and peak detection to the FPGA, using hardware line buffers to crush the 18.8 ms morphological bottleneck (40% of runtime) down to a fraction of a millisecond.
 - Eliminate Memory Copy Overhead: Eradicate the 15 ms wasted on windowing and matrix conversion by streaming raw audio samples directly through the hardware datapath on the fly.
 - Keep Hashing on the CPU: Stream only the final peak (x, y) coordinates back to the host processor via DMA, leaving the string-heavy SHA1 hashing and JSON serialization on the CPU where it belongs.

FUNCTIONAL PROFILING

Can this function be accelerated?

Function	Accelerate?	Why?
stride_windows()	Maybe	Mostly memory copying/rearranging. Simple loops, but likely memory-bandwidth bound rather than compute-bound. Not our priority.
detrend()	Maybe	Two passes over the data (compute mean, subtract mean). Easy to pipeline, but computationally light. Probably not worth offloading alone.
create_window()	No	Executed once. Generates the Hann coefficients. Could even be precomputed offline.
apply_window()	Yes	Pure element-wise multiplication. Independent iterations. Excellent HLS kernel (PIPELINE, vectorization).
get_sha1()	No	SHA-1 plus string manipulation. Small workload and lots of control logic. Poor FPGA target here.
generate_hashes()	No	Sorting, strings, JSON, SHA-1. Very software-oriented with dynamic memory. Leave on the CPU.
get_2D_peaks()	Yes (best candidate)	Replace the OpenCV morphology with your own sliding-window max filter. Great HLS candidate for pipelining and unrolling.
max_filter()	Yes	Same reason as above. If we re-write it instead of using opencv, it's almost tailor-made for HLS optimisations.
fingerprint()	Maybe (if we split it)	Can't accelerate the whole function but we can split it into kernels like FFT, windowing, and peak detection and leave orchestration on the CPU.
main()	No	File I/O, FFmpeg, printing. Definitely CPU.

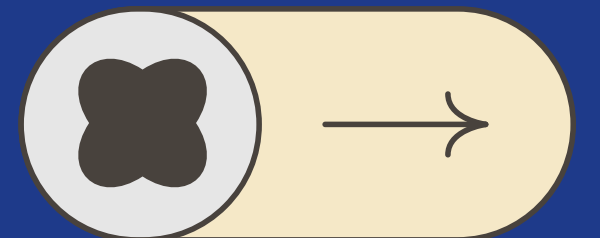
PROJECT SPECIFIC AIMS

1. Port and adapt the existing C++ Dejavu implementation as our software baseline (windowing → FFT → peak detection → hashing), replacing the OpenCV `cv::dilate` dependency with a custom 2D max-filter for peak detection.
2. Accelerate `get_2D_peaks()`, `max_filter()`, and `apply_window()` using Vitis HLS as separate hardware kernels on the PL, with `stride_windows()`, `detrend()`, and a split-out `fingerprint()` orchestration as further candidates if time allows.
3. Deploy the accelerated pipeline on the Kria KV260, integrating the HLS cores with the PS via AXI-Stream.
4. Quantify the CPU vs FPGA performance (execution time, and resource utilisation).



RISKS AND CONTINGENCIES

Risk	Mitigation
HLS FFT IP library fails / is incompatible with our Vitis HLS version	Drop FFT hardware acceleration entirely; focus remaining HLS effort on optimising the 2D peak detection core
Insufficient time to accelerate both FFT and peak detection cores	If time-constrained, prioritise and optimise the 2D peak detection core only



DESIGN

BASLINE ALGORITHM CONSTANTS

Constant	Value
DEFAULT_FAN_VALUE	5
MIN_HASH_TIME_DELTA	0
MAX_HASH_TIME_DELTA	100
FINGERPRINT_REDUCTION	20
PEAK_NEIGHBORHOOD_SIZE	5
DEFAULT_AMP_MIN	5
DEFAULT_WINDOW_SIZE	512
DEFAULT_OVERLAP_RATIO	0.25
FS (sample rate)	44100.0

KEY DESIGN DECISION

Removed OpenCV dependency (cv::dilate)

- cv::dilate is a black box — hard to manage and reason about for HLS.
- Implemented a custom 2D max-filter for peak detection instead.
- Gives explicit control over loop structure, needed for pragma-driven HLS optimisation.

PROGRESS

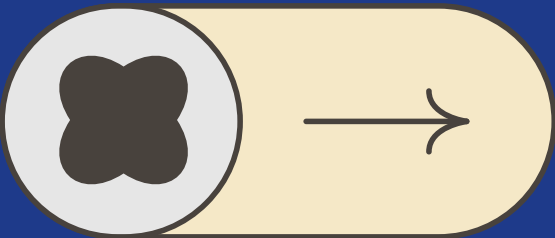
- Studied an existing implementation & identified major pipeline stages.
- Assessed suitability of FFT and peak detection for FPGA acceleration.
- Began profiling to locate computational bottlenecks.



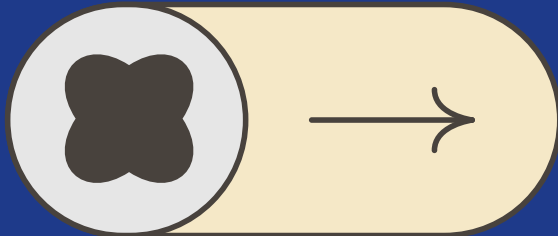
ROLES OF TEAM MEMBERS

Member	Key Responsibiities
Esha Tripathi	Porting and adapting the Dejavu C++ pipeline (windowing → FFT → peak detection → hashing) as the CPU baseline; profiling with std::chrono on both laptop and KV260 PS; test harness and Pvython reference vectors for verifving HLS kernel
Netik Maheshwari	System integration on the KV260 (PS–PL handoff); bitstream generation and Vivado implementation runs; performance and resource-utilisation benchmarking; project management and report writing
Rukhsaar Aujla	HLS apply_window kernel; stretch-goal FFT core; AXI-Stream interface design between PS and PL; Vivado block design and DMA/streaming integration
Venus Baranwal	HLS peak detection core (rewriting get_2D_peaks() / max_filter() to replace cv::dilate); pipelining, dataflow, and array-partitioning optimisation; fixed-point precision tuning for the sliding-window max filter

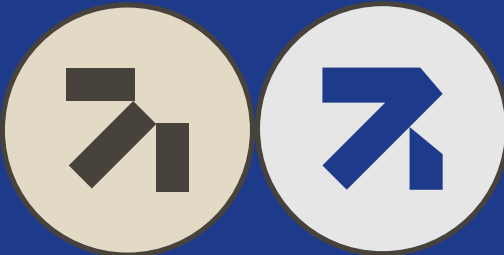
The project team is structured to ensure clarity and accountability. Each member has defined tasks that contribute to the overall project success.



Schedule



Task / Deliverable	W3	W4	W5	W6	W7	W8	W9	W10
Milestone: Team formation & project page	◆							
Algorithm study & literature review	■	■						
C/C++ software implementation		■	■					
Kria PS deployment & profiling			■	■				
Milestone: Project plan (this document)			◆					
Code partitioning for HLS				■	■			
HLS FFT core (baseline + synth)				■	■			
HLS peak detection core					■	■		
HLS optimisation (pipeline/dataflow)						■	■	
AXI interface & DMA integration						■	■	
Vivado block design & bitstream							■	■
Performance & energy benchmarking							■	■
Final report writing							■	■
Milestone: Presentation & demo							◆	◆
Milestone: Report & repo submitted								◆



THANK YOU

